ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA [[tên khoa — SV kiểm tra]]

BÁO CÁO ĐỒ ÁN CUỐI KỲ

MÔN: PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN (CS112)

ĐỀ TÀI

MÔ PHỎNG TRỰC QUAN TỪNG BƯỚC THUẬT TOÁN DIJKSTRA

so sánh cấu trúc dữ liệu (min-heap, mảng) và đối chiếu với Bellman-Ford

Sinh viên thực hiện: [Họ và tên]
MSSV: [MSSV]
Lớp: [CS112.xxxx]
Giảng viên hướng dẫn: [Tên giảng viên]
Mã nguồn: https://github.com/HoangLiem-AI/VisualDijkstra
Bản chạy online: https://hoangliem-ai.github.io/VisualDijkstra/
Video demo: [link video]

Thành phố Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

Ghi chú về bản nháp này 4
Chương 1. Giới thiệu 5
1.1. Bài toán đường đi ngắn nhất một nguồn 5
1.2. Mục tiêu đồ án 5
1.3. Phạm vi 5
1.4. Sản phẩm nộp 5
1.5. Bố cục báo cáo 6
Chương 2. Cơ sở lý thuyết 7
2.1. Ba chiến lược giải cùng một bài toán 7
2.2. Thuật toán Dijkstra 7
2.3. Min-heap nhị phân và lazy deletion 8
2.4. Bellman-Ford (quy hoạch động) 8
2.5. Vì sao Dijkstra sai khi có cạnh âm 9
2.6. So sánh tổng hợp 9
Chương 3. Phân tích và thiết kế 10
3.1. Yêu cầu chức năng 10
3.2. Kiến trúc: lõi sinh trace, giao diện chỉ phát lại 10
3.3. Cấu trúc một bước trace 11
3.4. Hàm relax(edge, t) 11
3.5. Định dạng input/output và xử lý lỗi 11
3.6. Các quyết định thiết kế 12
Chương 4. Cài đặt và giao diện 13
4.1. Vòng lặp chính của Dijkstra (min-heap) 13
4.2. Min-heap: thao tác sift-down 13
4.3. Giao diện 14
4.4. Cách chạy 15
Chương 5. Ví dụ minh họa thống nhất 16
5.1. Đồ thị ví dụ 16

5.2. Dijkstra (min-heap): toàn bộ 23 bước 16
5.3. Các bước đáng chú ý 17
5.4. Cùng đồ thị với Dijkstra mảng và Bellman-Ford 21
5.5. Cạnh âm và chu trình âm 23
Chương 6. Kiểm thử 26
6.1. Chiến lược 26
6.2. Bộ testcase 26
6.3. Cách chấm 26
6.4. Kết quả 26
6.5. Bộ test có bắt được lỗi không? (mutation test) 27
Chương 7. Đánh giá hiệu năng 28
7.1. Thiết lập 28
7.2. Đồ thị thưa 28
7.3. Đồ thị dày 29
7.4. Nhận xét 29
Chương 8. Quá trình xây dựng và sử dụng AI 30
8.1. Công cụ và cách tổ chức làm việc 30
8.2. Dòng thời gian 30
8.3. Lỗi phát hiện và cách xử lý 31
8.4. Kết quả của AI được kiểm chứng như thế nào 31
8.5. Phân công giữa sinh viên và AI 32
8.6. Nhận xét về việc dùng AI 32
Chương 9. Kết luận và hướng phát triển 33
9.1. Kết quả đạt được 33
9.2. Hạn chế 33
9.3. Hướng phát triển 33
Tài liệu tham khảo 34

Ghi chú về bản nháp này

Bản nháp do Claude Code (model Claude Opus 5.5) soạn ngày 18/10/2026 theo yêu cầu của sinh viên, từ `DEVLOG.md`, mã nguồn, số liệu kiểm thử/benchmark và trace thật do chương trình sinh ra (không chép tay). Quy trình này được ghi ở mục #14 của DEVLOG.

Các chỗ **tô vàng** dạng [...] là phần sinh viên phải tự điền hoặc tự viết, đặc biệt là phần **kiểm chứng và nhận xét của sinh viên** ở Chương 8. Sau khi sửa xong: xóa trang ghi chú này, nhấn chuột phải vào mục lục → *Update Field* → *Update entire table*, rồi xuất PDF (*File* → *Save As* → *PDF*).

Chương 1. Giới thiệu

1.1. Bài toán đường đi ngắn nhất một nguồn

Cho đồ thị có hướng có trọng số G = (V, E), hàm trọng số w: E → $\mathbb{R}$ và một đỉnh nguồn s. Độ dài của đường đi p = ⟨v_0, v_1, …, v_k⟩ là tổng trọng số các cạnh trên đường đi. Khoảng cách ngắn nhất δ(s, v) là giá trị nhỏ nhất của độ dài các đường đi từ s tới v (bằng ∞ nếu không có đường đi). Bài toán yêu cầu tính δ(s, v) cho mọi đỉnh v, và nếu có đỉnh đích t thì chỉ ra một đường đi ngắn nhất từ s tới t. Để gọn, trong báo cáo V và E cũng dùng để chỉ số đỉnh và số cạnh, chính là n và m ở dòng đầu input.

Đây là bài toán nền tảng trong định tuyến mạng, bản đồ, lập lịch vận chuyển. Ví dụ trực tiếp với sinh viên là tìm hành trình nhanh nhất cho bưu gửi giữa các bưu cục — hướng phát triển được nêu ở Chương 9.

1.2. Mục tiêu đồ án

- Xây dựng chương trình có giao diện cho phép **nhập input → theo dõi từng bước thuật toán → xem output**.
- Mô phỏng thuật toán Dijkstra dùng min-heap: tô sáng dòng pseudo-code đang chạy, trạng thái hàng đợi ưu tiên, bảng dist/prev và đồ thị tại mỗi bước; có thể tiến, lùi, tự chạy, nhảy tới bước bất kỳ.
- So sánh cách cài đặt (min-heap và mảng) và chiến lược thiết kế (tham lam – Dijkstra, quy hoạch động – Bellman-Ford), kể cả trên đồ thị có cạnh âm nơi Dijkstra cho kết quả sai.
- Chứng minh tính đúng bằng bộ testcase có đáp án độc lập, và đo thời gian thực tế để đối chiếu với độ phức tạp lý thuyết.

1.3. Phạm vi

Bảng 1.1. *Phạm vi đồ án*

Mức	Nội dung
Bắt buộc	Dijkstra dùng min-heap (lazy deletion), mô phỏng từng bước.
Mở rộng	Dijkstra dùng mảng $O(V^2)$; Bellman-Ford (quy hoạch động); chế độ minh họa cạnh âm (Dijkstra sai, Bellman-Ford đúng); phát hiện và tô chu trình âm.
Không làm	A*, bản đồ thật, hành trình đường thư theo lịch chuyến (để ở Hướng phát triển).

1.4. Sản phẩm nộp

Bảng 1.2. *Cấu trúc sản phẩm nộp*

Thư mục	Nội dung
`1_SourceCode/`	Mã nguồn: giao diện web (HTML, CSS, JavaScript ES modules,

Thư mục	Nội dung
	Cytoscape.js), lõi thuật toán, script Node.
`2_VideoDemo/`	Video demo và kịch bản quay `kich-ban-demo.md`.
`3_Testcase/`	44 test chia 5 nhóm, mỗi test gồm `.in` và `.out`; `README.md` mô tả từng test.
`4_Report/`	Báo cáo (Word + PDF), hình, số liệu kiểm thử và benchmark.
`DEVLOG.md`	Nhật ký phát triển: prompt nguyên văn, kết quả AI, lỗi, quyết định thiết kế.

Yêu cầu **thống nhất** được đảm bảo bằng một đồ thị duy nhất 6 đỉnh, 11 cạnh: nó là ví dụ ở Chương 5 của report, là ví dụ được quay trong video, và là testcase `3_Testcase/nho/01-vi-du-6-dinh`. Đồ thị này được khai báo một lần trong `src/core/samples.js` và mọi nơi khác đều lấy từ đó.

Truy cập sản phẩm. Bản chạy online đặt trên GitHub Pages: https://hoangliem-ai.github.io/VisualDijkstra/ — mở bằng trình duyệt, không cần cài đặt hay đăng nhập. Mã nguồn, bộ testcase, DEVLOG và report nằm trong repo công khai https://github.com/HoangLiem-AI/VisualDijkstra. Mọi link trong báo cáo đều để chế độ công khai, không cần xin quyền truy cập và không có thời hạn. Nếu không mở được bản online, có thể chạy trên máy theo mục cài đặt trong README.md (chỉ cần Node.js).

1.5. Bố cục báo cáo

Chương 2 trình bày cơ sở lý thuyết. Chương 3 và 4 trình bày thiết kế và cài đặt. Chương 5 chạy từng bước trên ví dụ thống nhất. Chương 6 và 7 là kiểm thử và đánh giá hiệu năng. Chương 8 tường thuật quá trình xây dựng, trong đó có cách sử dụng AI. Chương 9 là kết luận và hướng phát triển.

Chương 2. Cơ sở lý thuyết

2.1. Ba chiến lược giải cùng một bài toán

Bảng 2.1. *Các chiến lược thiết kế được dùng trong đồ án*

Chiến lược	Thuật toán	Ý tưởng	Độ phức tạp
Vét cạn	Duyệt mọi đường đi đơn (DFS)	Thử hết, lấy đường ngắn nhất	O(n!) — chỉ dùng làm đáp án chuẩn với n ≤ 8
Tham lam	Dijkstra	Mỗi lần chốt đỉnh có dist tạm thời nhỏ nhất	O((V + E) log V) với heap; O(V^2 + E) với mảng
Quy hoạch động	Bellman-Ford	Tính dần đường đi ngắn nhất dùng ≤ k cạnh	O(V · E)

Cả ba đều dựa trên **cấu trúc con tối ưu**: một đoạn con của đường đi ngắn nhất cũng là đường đi ngắn nhất. Phép toán cơ bản chung là **relax** cạnh (u, v): nếu dist[u] + w(u, v) < dist[v] thì cập nhật dist[v] và prev[v] = u. Các thuật toán chỉ khác nhau ở *thứ tự* relax các cạnh.

2.2. Thuật toán Dijkstra

Ý tưởng

Dijkstra duy trì tập S các đỉnh đã **chốt** (dist đã là khoảng cách ngắn nhất). Mỗi bước chọn đỉnh u ∉ S có dist[u] nhỏ nhất, đưa u vào S, rồi relax mọi cạnh ra khỏi u. Đây là lựa chọn tham lam: đỉnh gần nhất trong số chưa chốt được coi là đã biết khoảng cách chính xác.

Pseudo-code (bản min-heap, đúng như hiển thị trên giao diện)

```
 1  for mỗi đỉnh v: dist[v] ← ∞; prev[v] ← null
 2  dist[s] ← 0; PQ.push(s, 0)
 3  while PQ khác rỗng:
 4      (u, d) ← PQ.popMin()
 5      if u đã chốt: continue          ▷ phần tử cũ (lazy deletion)
 6      chốt u                          ▷ dist[u] là ngắn nhất
 7      for mỗi cạnh (u, v, w) với v chưa chốt:
 8          nd ← relax((u, v, w), dist[u])   ▷ mặc định dist[u] + w
 9          if nd < dist[v]:
10              dist[v] ← nd; prev[v] ← u; PQ.push(v, nd)
11  return dist, prev
```

Tính đúng

Mệnh đề. Nếu mọi trọng số không âm thì khi một đỉnh u được chốt, dist[u] = δ(s, u).

Chứng minh (phản chứng). Giả sử u là đỉnh *đầu tiên* được chốt mà dist[u] > δ(s, u). Khi đó u ≠ s và tồn tại đường đi ngắn nhất P từ s tới u. Gọi y là đỉnh đầu tiên trên P chưa được chốt tại thời điểm chốt u, và x là đỉnh đứng ngay trước y trên P (x đã chốt). Vì x được chốt trước u nên dist[x] = δ(s, x), và khi chốt x cạnh (x, y) đã được relax, do đó dist[y] ≤ δ(s, x) + w(x, y) = δ(s, y). Vì y nằm trên P trước u và **các trọng số không âm**, δ(s, y) ≤ δ(s, u). Suy ra dist[y] ≤ δ(s, u) < dist[u], trái với việc u được chọn vì có dist nhỏ nhất. ■

Chỗ dùng giả thiết "trọng số không âm" chính là chỗ Dijkstra hỏng khi có cạnh âm (mục 2.5).

Độ phức tạp

- **Bản mảng:** V lần tìm đỉnh nhỏ nhất, mỗi lần duyệt mảng O(V); mỗi cạnh relax một lần. Tổng **O(V² + E)**.
- **Bản min-heap với lazy deletion:** mỗi lần relax thành công đẩy một phần tử mới vào heap (không có decrease-key), nên heap có tối đa E + 1 phần tử. Mỗi push/pop tốn O(log E) = O(log V) (vì E ≤ V²). Tổng **O((V + E) log V)**, bộ nhớ O(V + E).
- Đồ thị thưa (E ≈ V): bản heap O(V log V) tốt hơn hẳn O(V²). Đồ thị dày (E ≈ V²): bản heap O(V² log V), bản mảng O(V²) lại tốt hơn. Chương 7 kiểm chứng điều này bằng số đo.

2.3. Min-heap nhị phân và lazy deletion

Min-heap nhị phân là cây nhị phân đầy đủ lưu trong mảng: nút i có con 2i + 1, 2i + 2 và cha ⌊(i − 1)/2⌋; khóa của cha luôn ≤ khóa của con nên phần tử nhỏ nhất ở gốc. `push` đặt phần tử vào cuối rồi đẩy lên (sift-up), `pop` đưa phần tử cuối lên gốc rồi đẩy xuống (sift-down); cả hai O(log n).

Dijkstra cần giảm khóa của một đỉnh khi relax thành công (decrease-key). Thay vì cài decrease-key (phải lưu vị trí của mỗi đỉnh trong heap), đồ án dùng **lazy deletion**: đẩy luôn cặp (v, dist mới) vào heap; cặp cũ vẫn nằm lại và sẽ bị **bỏ qua khi lấy ra** nếu đỉnh đã chốt. Trên giao diện, bước bỏ qua này có loại `skip-stale`, và phần tử cũ trong hàng đợi được gạch ngang để người xem thấy rõ.

Khi hai khóa bằng nhau, heap ưu tiên đỉnh có số nhỏ hơn. Nhờ vậy trace luôn tái tạo được và bản heap, bản mảng chốt các đỉnh theo cùng thứ tự.

2.4. Bellman-Ford (quy hoạch động)

Gọi d_k(v) là độ dài đường đi ngắn nhất từ s tới v dùng **tối đa k cạnh**. Công thức truy hồi:

$$d_k(v) = \min\left(d_{k-1}(v), \min_{(u,v)\in E} [\, d_{k-1}(u) + w(u, v) \,] \right), \quad d_0(s) = 0,$$
$$d_0(v) = \infty \text{ với } v \neq s.$$

Nếu không có chu trình âm, đường đi ngắn nhất là đường đi đơn, dùng tối đa $V - 1$ cạnh, nên sau $V - 1$ vòng (mỗi vòng relax mọi cạnh) ta có kết quả đúng. Thêm một vòng nữa: nếu vẫn còn cạnh relax được thì tồn tại chu trình âm tới được từ s và bài toán không có lời giải; chương trình lần theo prev để chỉ ra chu trình đó. Độ phức tạp $O(V \cdot E)$. Cài đặt có **dừng sớm**: nếu một vòng không cập nhật gì thì các vòng sau cũng không, nên dừng.

Có thể xem Dijkstra cũng là quy hoạch động trên cùng cấu trúc con tối ưu, nhưng thứ tự giải các bài toán con được chọn bằng tham lam (đỉnh gần nhất trước), nhờ đó mỗi cạnh chỉ cần relax một lần.

2.5. Vì sao Dijkstra sai khi có cạnh âm

Xét đồ thị 4 đỉnh: 1→2 (2), 1→3 (5), 3→2 (−4), 2→4 (3), nguồn s = 1. Dijkstra chốt đỉnh 2 với dist = 2 ngay sau đỉnh 1, vì 2 là nhỏ nhất. Nhưng đường 1→3→2 dài 5 + (−4) = 1 < 2. Khi xét đỉnh 3 thì đỉnh 2 đã chốt, cạnh 3→2 bị bỏ qua, và đỉnh 4 nhận dist = 5 thay vì 4. Kết quả Dijkstra: `0 2 5 5`; đúng (Bellman-Ford): `0 1 5 4`. Ví dụ này có sẵn trên giao diện (mục 5.5) và là testcase `dac-biet/01-canh-am-dijkstra-sai`.

Để minh họa được điều này, cài đặt giữ đúng Dijkstra kinh điển: kiểm tra phần tử cũ bằng "u đã chốt" và **không relax tới đỉnh đã chốt**. Nếu cho phép xử lý lại đỉnh, thuật toán trở thành dạng label-correcting và có thể tình cờ cho kết quả đúng, làm mất ý nghĩa minh họa. Ở chế độ bình thường, Dijkstra từ chối chạy và báo lỗi rõ ràng khi đồ thị có cạnh âm.

2.6. So sánh tổng hợp

Bảng 2.2. *So sánh ba thuật toán*

Tiêu chí	Dijkstra (min-heap)	Dijkstra (mảng)	Bellman-Ford
Chiến lược	Tham lam	Tham lam	Quy hoạch động
Thời gian	$O((V + E) \log V)$	$O(V^2 + E)$	$O(V \cdot E)$
Bộ nhớ	$O(V + E)$	$O(V)$	$O(V)$
Cạnh âm	Không (báo lỗi)	Không (báo lỗi)	Có
Chu trình âm	—	—	Phát hiện và chỉ ra
Phù hợp	Đồ thị thưa	Đồ thị dày	Có cạnh âm

Chương 3. Phân tích và thiết kế

3.1. Yêu cầu chức năng

- Nhập đồ thị dạng văn bản (kiểu Wecode), chọn có hướng / vô hướng, chọn thuật toán, chọn ví dụ có sẵn.
- Phát lại từng bước: Tiến, Lùi, Về đầu, Tới cuối, Tự chạy với tốc độ điều chỉnh được, thanh trượt nhảy tới bước bất kỳ, phím tắt.
- Mỗi bước hiển thị: đồ thị tô màu theo trạng thái, dòng pseudo-code đang chạy, hàng đợi ưu tiên, bảng dist/prev, câu giải thích tiếng Việt, nhật ký.
- Output đúng định dạng đề; input sai, cạnh âm (với Dijkstra), chu trình âm được báo lỗi rõ ràng, không crash.
- Đồ thị lớn (tới 10^5 đỉnh, $2 \cdot 10^5$ cạnh) chạy bằng script, không vẽ.

3.2. Kiến trúc: lõi sinh trace, giao diện chỉ phát lại

Quyết định quan trọng nhất là **tách lõi thuật toán khỏi giao diện**. Thuật toán chạy một lần, trả về `{ result, trace }`, trong đó `trace` là mảng các bước. Giao diện chỉ phát lại trace, không tự tính lại. Nhờ vậy:

- Cùng một mã thuật toán được giao diện web, script kiểm thử và script benchmark dùng chung; cái được kiểm thử chính là cái được mô phỏng.
- Lùi bước trở nên đơn giản (chỉ đổi chỉ số trong mảng trace), không phải "chạy ngược" thuật toán.
- Lõi không đụng tới DOM nên chạy được bằng Node.js.

Bảng 3.1. *Các module của chương trình*

Tầng	File	Vai trò
Lõi (`src/core/`)	`parser.js`	Đọc và kiểm tra input, báo lỗi kèm số dòng; định dạng output.
	`graph.js`	Danh sách kề, đường đi từ prev, hàm `relax` mặc định, kiểm tra cạnh âm.
	`min-heap.js`	Heap nhị phân, khóa bằng nhau thì đỉnh nhỏ trước.
	`dijkstra-heap.js`, `dijkstra-array.js`, `bellman-ford.js`	Ba thuật toán, cùng trả về `{ result, trace }`.
	`algorithms.js`, `samples.js`, `solve.js`	Danh mục thuật toán; đồ thị ví dụ dùng chung; chuỗi input → chuỗi output.
Giao diện (`src/ui/`)	`renderer.js`	Vẽ đồ thị bằng Cytoscape.js, tô màu theo bước.
	`player.js`	Tiến / Lùi / Tự chạy (không đụng DOM).
	`panels.js`, `app.js`	Pseudo-code, hàng đợi, bảng dist, nhật ký; nối các phần.
Script (`scripts/`)	`gen-tests.js`, `run-tests.js`, `brute-force.js`, `benchmark.js`	Sinh testcase có seed, chấm, đáp án vét cạn/Floyd, đo thời gian.

3.3. Cấu trúc một bước trace

Bảng 3.2. *Các trường của một bước trace*

Trường	Ý nghĩa
`type`	`init`, `extract`, `skip-stale`, `relax-ok`, `relax-fail`, `done`; Bellman-Ford thêm `round`, `negative-cycle`.
`line`	Số dòng pseudo-code đang chạy, để tô sáng.
`current`, `edge`	Đỉnh đang xét và cạnh đang relax.
`dist`, `prev`, `finalized`, `pq`	**Bản sao** tại thời điểm đó (không phải tham chiếu), nên lùi bước luôn đúng.
`message`	Câu giải thích tiếng Việt, ví dụ "Relax 3→2: 4 → 3".
`round`, `maxRound`, `cycle`	Chỉ Bellman-Ford: vòng hiện tại và chu trình âm.

Vì mỗi bước sao chép mảng O(V), trace tốn O(số bước × V) bộ nhớ. Điều này chấp nhận được với đồ thị ≤ 30 đỉnh trên giao diện; khi chạy đồ thị lớn, script truyền `recordTrace: false` để tắt ghi trace.

3.4. Hàm relax(edge, t)

Mọi thuật toán tính giá trị mới qua `relax(edge, t)` (mặc định trả về `t + edge.w`) thay vì cộng trực tiếp trong vòng lặp. Lý do: hướng phát triển là bài toán hành trình đường thư, trong đó cạnh là các chuyến có giờ đi/giờ đến; khi đó chỉ cần thay hàm relax bằng "giờ đến sớm nhất nếu có mặt ở u lúc t" mà không sửa thuật toán.

3.5. Định dạng input/output và xử lý lỗi

```
n m s [t]          ▷ số đỉnh, số cạnh, đỉnh nguồn, đỉnh đích (không bắt buộc)
u1 v1 w1          ▷ m dòng cạnh, đỉnh đánh số 1..n
...
um vm wm
```

- Ràng buộc: $1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$; $|w| \le 10^9$ (Dijkstra yêu cầu $w \ge 0$). Tổng lớn nhất khoảng $10^{14} < 2^{53}$ nên kiểu `Number` của JavaScript vẫn chính xác.
- Output: n dòng dist (−1 nếu không tới được); nếu có t thì thêm dòng đường đi `s -> ... -> t` (−1 nếu không có đường).
- Chu trình âm (Bellman-Ford): một dòng `Phát hiện chu trình âm`. Cạnh âm với Dijkstra: thông báo lỗi kèm gợi ý dùng Bellman-Ford.
- Input sai: thông báo kèm số dòng, ví dụ `Dòng 4: "x" không phải số nguyên (w)` (Hình 4.2).

3.6. Các quyết định thiết kế

Bảng dưới lấy từ mục "Các quyết định thiết kế" trong DEVLOG. Các quyết định do AI đề xuất được đánh dấu để sinh viên xác nhận.

Bảng 3.3. *Các quyết định thiết kế chính*

Quyết định	Phương án đã cân nhắc	Lý do
Phạm vi: Dijkstra heap + mảng + Bellman-Ford	Chỉ Dijkstra; thêm A* và bản đồ thật	Cân bằng sáng tạo và thời gian (hạn 18/10)
Web tĩnh + Cytoscape.js; script bằng Node	Python (Streamlit/PyQt); .NET	Vẽ đồ thị tốt, chạy trên trình duyệt, dùng chung lõi
Lazy deletion, kiểm tra "u đã chốt"; không relax tới đỉnh đã chốt	Kiểm tra d > dist[u] và cho xử lý lại đỉnh	Giữ đúng Dijkstra kinh điển để minh họa sai với cạnh âm
Parser cho phép w âm; Dijkstra tự báo lỗi	Parser cấm cạnh âm	Dùng chung parser cho Bellman-Ford
Đồ thị ví dụ để trong `samples.js`	Mỗi nơi tự chép đồ thị	Report, video, testcase dùng đúng một đồ thị
Bellman-Ford duyệt cạnh theo thứ tự input, dừng sớm	Luôn chạy đủ V − 1 vòng	Trace khớp file input, ngắn hơn khi demo
Chấm đường đi bằng kiểm tra hợp lệ + độ dài	So khớp chuỗi	Có thể có nhiều đường ngắn nhất
Đáp án test lớn bằng Bellman-Ford	Dùng Dijkstra làm đáp án	Nguồn đáp án độc lập với thuật toán được kiểm tra

Chương 4. Cài đặt và giao diện

4.1. Vòng lặp chính của Dijkstra (min-heap)

Đoạn mã dưới trích từ `src/core/dijkstra-heap.js` (đã lược phần đếm thống kê). Mỗi nhánh gọi `record(...)` để ghi một bước trace với số dòng pseudo-code tương ứng.

```
while (!pq.isEmpty()) {
  // Dòng 4: lấy phần tử có khóa nhỏ nhất
  const { vertex: u, key: d } = pq.pop();
  // Dòng 5: lazy deletion – u đã chốt thì (u, d) là bản cũ, bỏ qua
  if (finalized[u]) {
    record('skip-stale', 5, u, null, `Lấy (${u}, ${d}) nhưng đỉnh ${u} đã chốt
...`);
    continue;
  }
  // Dòng 6: chốt u, dist[u] không thay đổi nữa
  finalized[u] = true;
  record('extract', 6, u, null, `Lấy (${u}, ${d}) ra khỏi hàng đợi, chốt dist[${u}]
= ${d}`);
  // Dòng 7: duyệt các cạnh ra khỏi u
  for (const edge of graph.adj[u]) {
    const v = edge.v;
    if (finalized[v]) { record('relax-fail', 7, u, edge, `Bỏ qua ${u}→${v}: đỉnh
${v} đã chốt`); continue; }
    // Dòng 8: tính giá trị mới qua hàm relax (không cộng trực tiếp)
    const nd = relax(edge, dist[u]);
    // Dòng 9–10: cập nhật nếu tốt hơn, đẩy bản mới vào heap
    if (nd < dist[v]) {
      dist[v] = nd; prev[v] = u; pq.push(v, nd);
      record('relax-ok', 10, u, edge, `Relax ${u}→${v}: ${old} → ${nd}`);
    } else {
      record('relax-fail', 9, u, edge, `Relax ${u}→${v}: ... ≥ ..., giữ nguyên`);
    }
  }
}
```

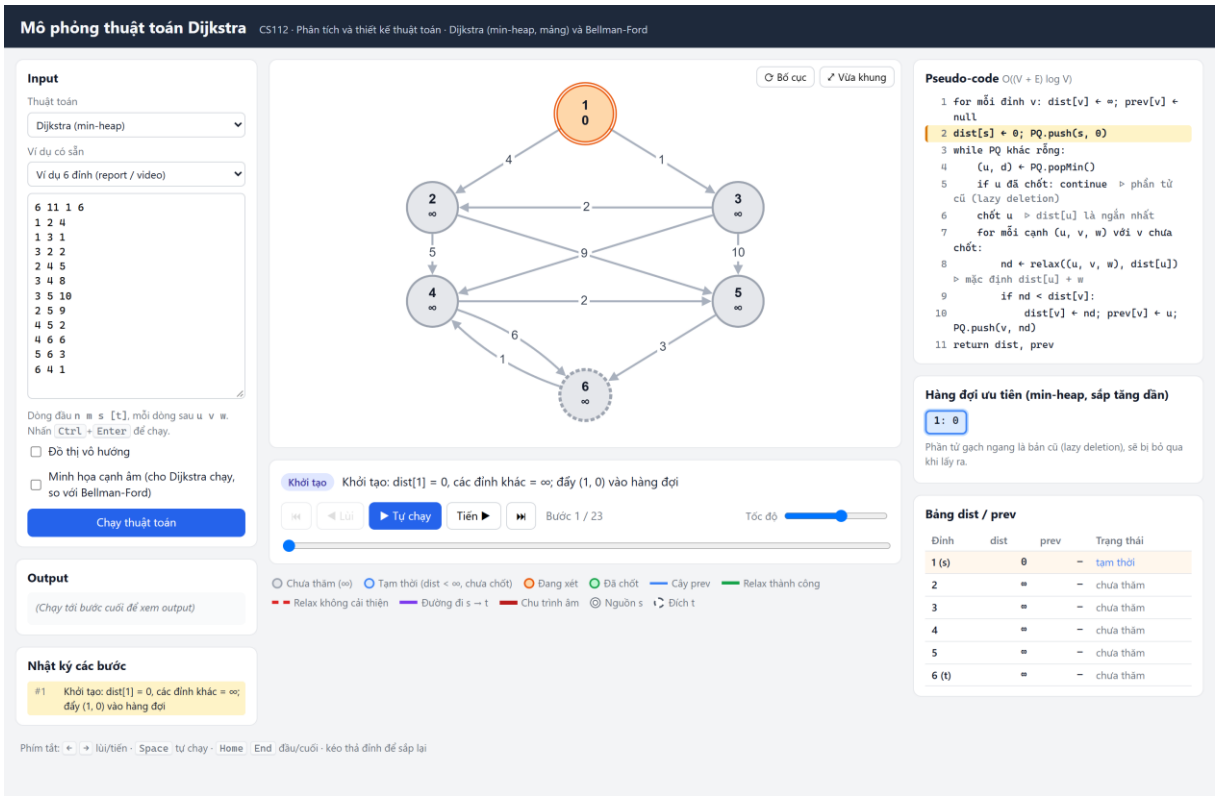
4.2. Min-heap: thao tác sift-down

```
#siftDown(i) {
  const n = this.items.length;
  for (;;) {
    const l = 2 * i + 1;
    const r = l + 1;
    let smallest = i;
    if (l < n && this.#less(l, smallest)) smallest = l;
    if (r < n && this.#less(r, smallest)) smallest = r;
    if (smallest === i) break;
    this.#swap(i, smallest);
    i = smallest;
  }
}
```

Dòng so sánh với con phải ở trên được dùng làm "lỗi cài cắm" khi kiểm tra chất lượng bộ test (mục 6.5).

4.3. Giao diện

Giao diện gồm ba cột (Hình 4.1). Cột trái: chọn thuật toán, ví dụ có sẵn, ô nhập input, tùy chọn vô hướng và minh họa cạnh âm, output và nhật ký các bước (bấm vào một dòng để nhảy tới bước đó). Cột giữa: đồ thị, câu giải thích của bước hiện tại, các nút điều khiển và chú thích màu. Cột phải: pseudo-code với dòng đang chạy được tô sáng, hàng đợi ưu tiên (Dijkstra mảng: danh sách đỉnh ứng viên; Bellman-Ford: thanh tiến độ vòng lặp) và bảng dist/prev.



Hình 4.1. *Giao diện tại bước 1 (khởi tạo) của ví dụ thống nhất*

Bảng 4.1. *Ý nghĩa màu trên đồ thị*

Màu / ký hiệu	**Ý nghĩa**
Đỉnh xám	Chưa thăm (dist = ∞)
Viền xanh dương	Tạm thời: dist < ∞, chưa chốt
Viền cam	Đỉnh đang xét
Nền xanh lá	Đã chốt
Cạnh xanh lá / đỏ nét đứt	Relax thành công / không cải thiện
Cạnh xanh dương	Cây prev (cạnh dẫn tới dist hiện tại)
Cạnh tím (bước cuối)	Đường đi ngắn nhất s → t
Cạnh và đỉnh đỏ	Chu trình âm
Phần tử gạch ngang trong hàng đợi	Phần tử cũ, sẽ bị bỏ qua (lazy deletion)

Phím tắt: ← → lùi/tiến, Space tự chạy, Home/End về đầu/cuối, Ctrl + Enter chạy thuật toán. Giao diện giới hạn 30 đỉnh, 150 cạnh để hình vẽ còn đọc được.



Hình 4.2. *Input sai được báo lỗi kèm số dòng, không crash*

4.4. Cách chạy

```
cd 1_SourceCode
node scripts/serve.js            ▷ mở http://localhost:8080
node scripts/demo-trace.js       ▷ in trace ra console
node scripts/run-tests.js        ▷ chấm toàn bộ 3_Testcase
node scripts/benchmark.js        ▷ đo thời gian
```

Không mở trực tiếp `index.html` bằng double-click vì trình duyệt chặn ES modules qua `file://`. Chương trình không có thư viện cần cài (Cytoscape.js tải qua CDN), chỉ cần Node.js. Chi tiết trong `README.md`.

Chương 5. Ví dụ minh họa thống nhất

5.1. Đồ thị ví dụ

Đồ thị có hướng 6 đỉnh, 11 cạnh, nguồn s = 1, đích t = 6. Đồ thị được chọn để trace có **đủ mọi loại bước**: relax thành công, relax không cải thiện, bỏ qua vì đỉnh đã chốt, và bỏ qua phần tử cũ trong heap. Cạnh `6 4 1` được thêm vào sau lần chạy đầu chính vì lý do này (lỗi #1 trong DEVLOG).

```
6 11 1 6
1 2 4
1 3 1
3 2 2
2 4 5
3 4 8
3 5 10
2 5 9
4 5 2
4 6 6
5 6 3
6 4 1
```

Output (giống nhau ở cả 3 thuật toán; trùng với `3_Testcase/nho/01-vi-du-6-dinh.out`):

```
0
3
1
8
10
13
1 -> 3 -> 2 -> 4 -> 5 -> 6
```

5.2. Dijkstra (min-heap): toàn bộ 23 bước

Bảng dưới được sinh tự động từ trace thật (`4_Report/tools/xuat-trace.mjs`), trùng khớp với bộ đếm "Bước k / 23" trên giao diện và trong video.

Bảng 5.1. *Trace Dijkstra (min-heap) trên ví dụ thống nhất*

#	Loại	Dòng	Diễn giải	Hàng đợi sau bước (tăng dần)
1	Khởi tạo	2	Khởi tạo: dist[1] = 0, các đỉnh khác = ∞; đẩy (1, 0) vào hàng đợi	(1, 0)
2	Lấy ra, chốt	6	Lấy (1, 0) ra khỏi hàng đợi, chốt dist[1] = 0	∅
3	Relax: cập nhật	10	Relax 1→2: ∞ → 4	(2, 4)
4	Relax: cập nhật	10	Relax 1→3: ∞ → 1	(3, 1) (2, 4)
5	Lấy ra, chốt	6	Lấy (3, 1) ra khỏi hàng đợi, chốt dist[3] = 1	(2, 4)
6	Relax: cập nhật	10	Relax 3→2: 4 → 3	(2, 3) (2, 4)
7	Relax: cập nhật	10	Relax 3→4: ∞ → 9	(2, 3) (2, 4) (4, 9)
8	Relax: cập nhật	10	Relax 3→5: ∞ → 11	(2, 3) (2, 4) (4, 9) (5, 11)
9	Lấy ra, chốt	6	Lấy (2, 3) ra khỏi hàng đợi, chốt dist[2] = 3	(2, 4) (4, 9) (5, 11)
10	Relax: cập nhật	10	Relax 2→4: 9 → 8	(2, 4) (4, 8) (4, 9) (5, 11)
11	Relax: không đổi	9	Relax 2→5: 3 + 9 = 12 ≥ 11, giữ nguyên	(2, 4) (4, 8) (4, 9) (5, 11)

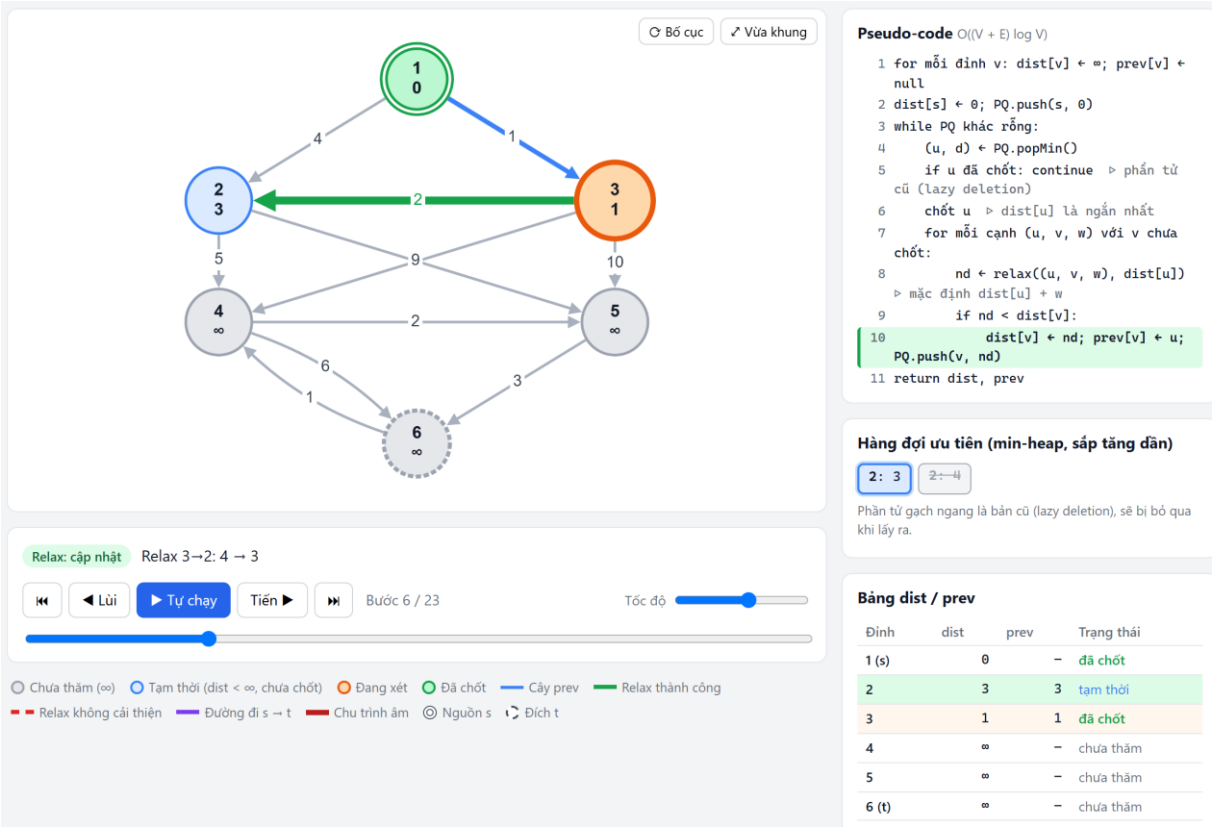
#	Loại	Dòng	Diễn giải	Hàng đợi sau bước (tăng dần)
12	Bỏ qua bản cũ	5	Lấy (2, 4) nhưng đỉnh 2 đã chốt với dist = 3 → bỏ qua phần tử cũ	(4, 8) (4, 9) (5, 11)
13	Lấy ra, chốt	6	Lấy (4, 8) ra khỏi hàng đợi, chốt dist[4] = 8	(4, 9) (5, 11)
14	Relax: cập nhật	10	Relax 4→5: 11 → 10	(4, 9) (5, 10) (5, 11)
15	Relax: cập nhật	10	Relax 4→6: ∞ → 14	(4, 9) (5, 10) (5, 11) (6, 14)
16	Bỏ qua bản cũ	5	Lấy (4, 9) nhưng đỉnh 4 đã chốt với dist = 8 → bỏ qua phần tử cũ	(5, 10) (5, 11) (6, 14)
17	Lấy ra, chốt	6	Lấy (5, 10) ra khỏi hàng đợi, chốt dist[5] = 10	(5, 11) (6, 14)
18	Relax: cập nhật	10	Relax 5→6: 14 → 13	(5, 11) (6, 13) (6, 14)
19	Bỏ qua bản cũ	5	Lấy (5, 11) nhưng đỉnh 5 đã chốt với dist = 10 → bỏ qua phần tử cũ	(6, 13) (6, 14)
20	Lấy ra, chốt	6	Lấy (6, 13) ra khỏi hàng đợi, chốt dist[6] = 13	(6, 14)
21	Relax: không đổi	7	Bỏ qua 6→4: đỉnh 4 đã chốt	(6, 14)
22	Bỏ qua bản cũ	5	Lấy (6, 14) nhưng đỉnh 6 đã chốt với dist = 13 → bỏ qua phần tử cũ	∅
23	Kết thúc	11	Hàng đợi rỗng → kết thúc. Đường đi ngắn nhất 1→6: 1 → 3 → 2 → 4 → 5 → 6 (độ dài 13)	∅

Thống kê: 10 lần push, 10 lần pop, trong đó 4 phần tử cũ bị bỏ qua; 10 lần tính relax, 9 lần cập nhật.

5.3. Các bước đáng chú ý

Bước 6 — relax làm giảm khoảng cách

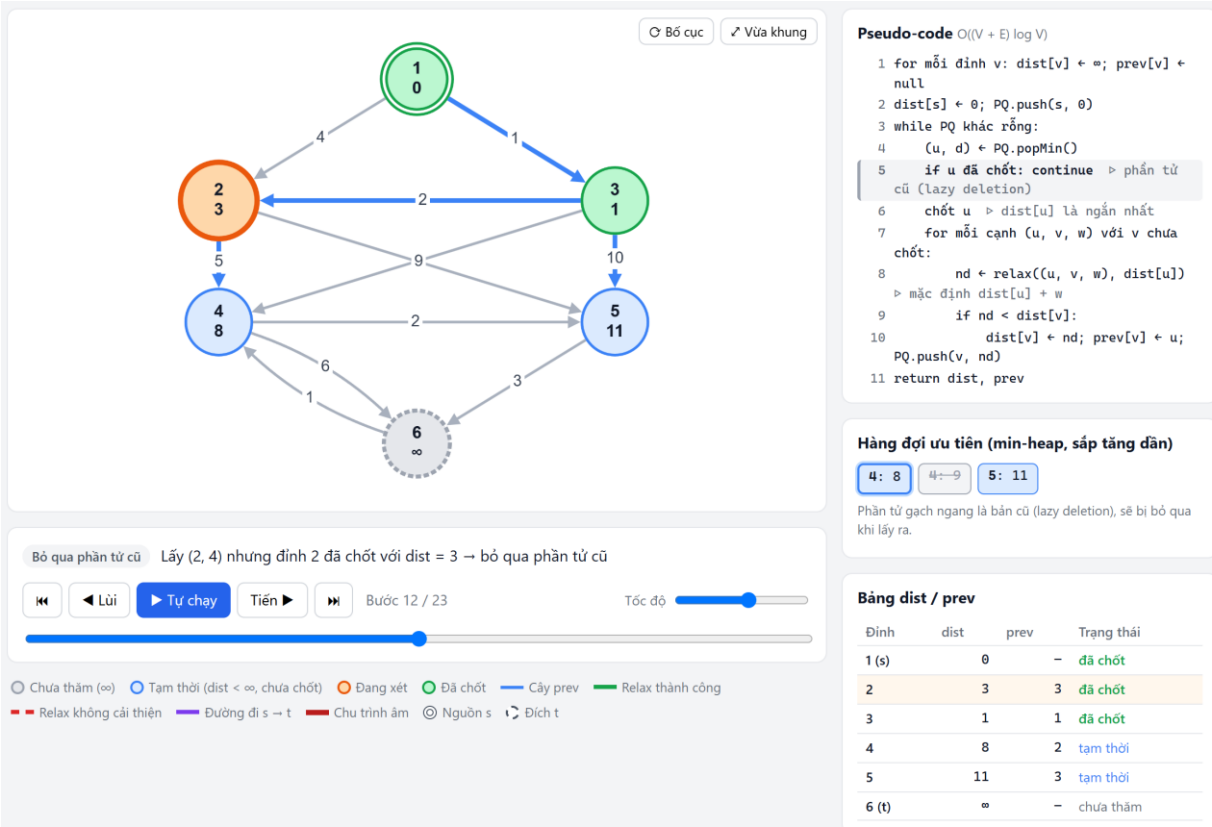
Đỉnh 3 vừa được chốt với dist = 1. Relax cạnh 3→2 cho 1 + 2 = 3 < 4, nên dist[2] giảm từ 4 xuống 3, prev[2] = 3, và cặp (2, 3) được đẩy vào heap. Cặp cũ (2, 4) **không bị xóa**: nó vẫn nằm trong heap và được gạch ngang.



Hình 5.1. *Bước 6: relax 3→2, dist[2] giảm 4 → 3; (2, 4) trở thành phần tử cũ*

Bước 12 — lazy deletion

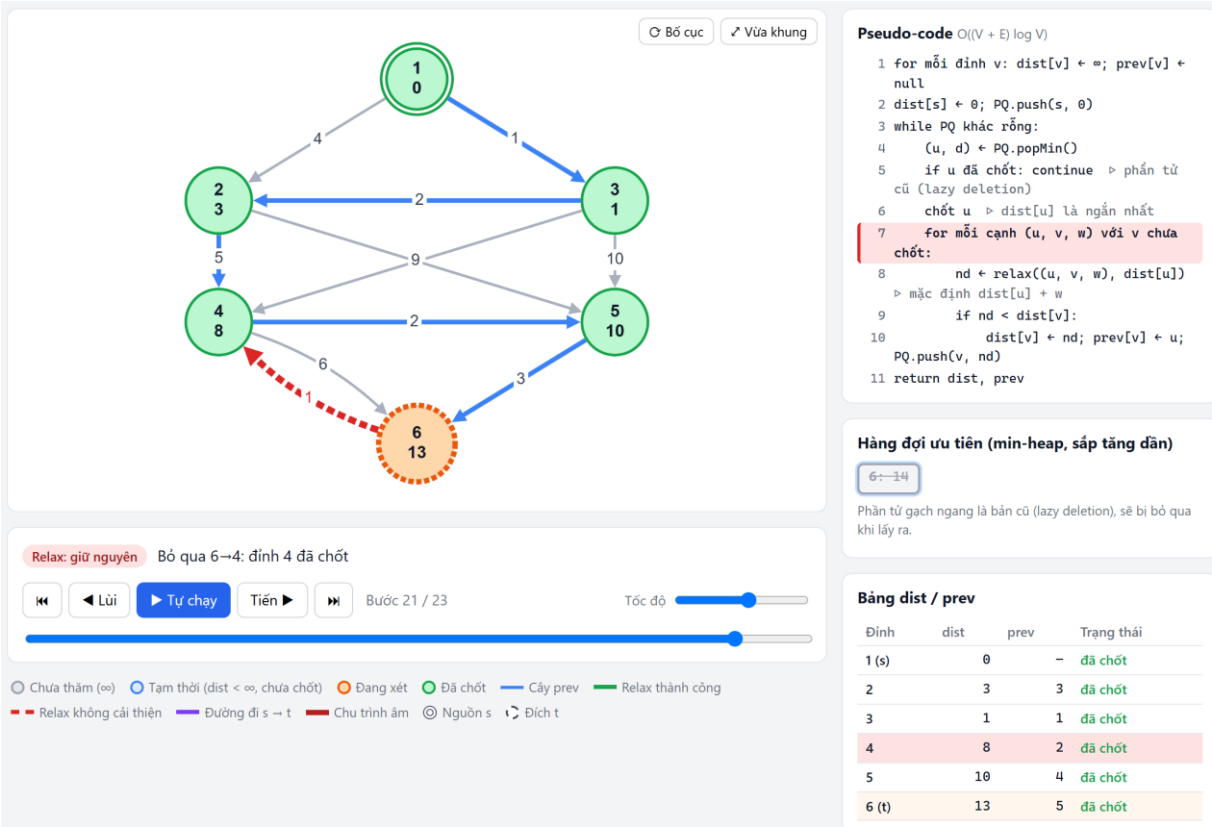
Heap trả về (2, 4), nhưng đỉnh 2 đã được chốt ở bước 9 với dist = 3, nên phần tử này bị bỏ qua (dòng 5 của pseudo-code). Đây là cái giá của lazy deletion: heap lớn hơn một chút (10 lần push cho 6 đỉnh), đổi lại không cần decrease-key.



Hình 5.2. *Bước 12: lấy ra phần tử cũ (2, 4) và bỏ qua*

Bước 21 — không relax tới đỉnh đã chốt

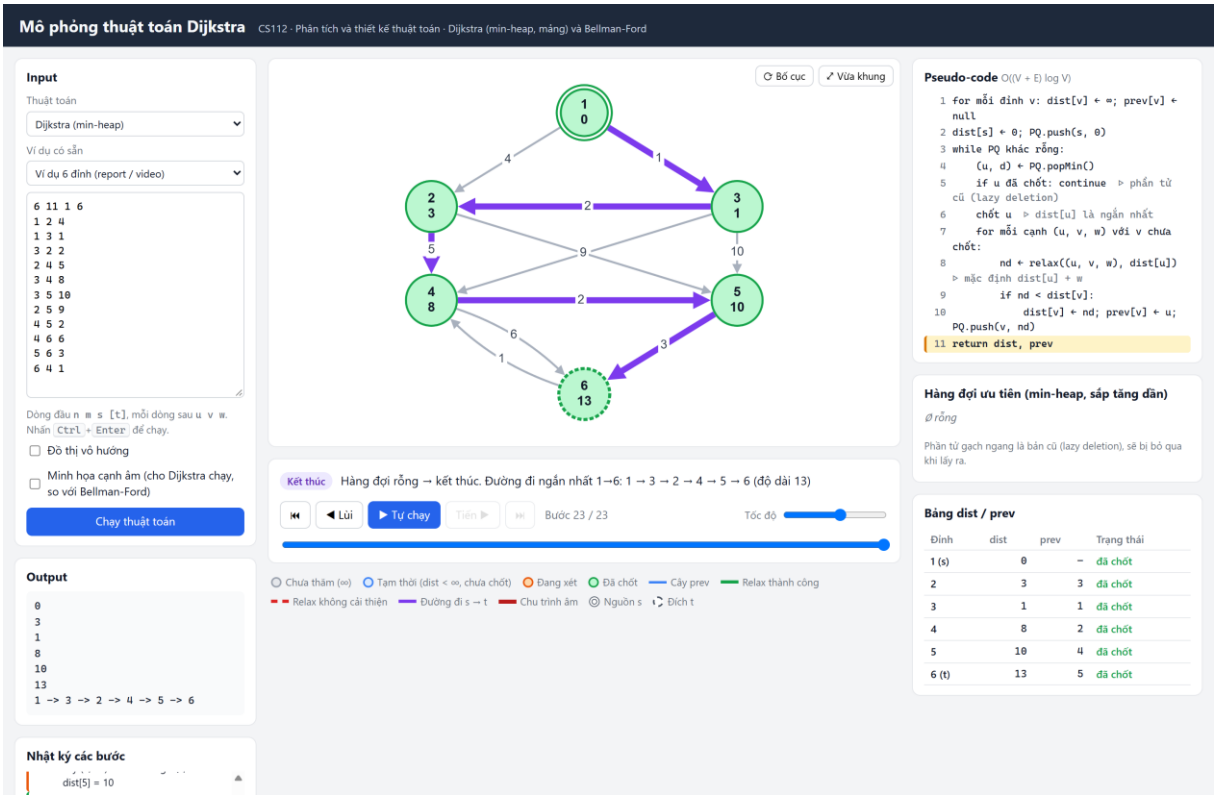
Đỉnh 6 được chốt với dist = 13; cạnh 6→4 dẫn tới đỉnh 4 đã chốt nên bị bỏ qua (dòng 7). Với trọng số không âm, việc này không bao giờ làm mất kết quả đúng (mục 2.2).



Hình 5.3. *Bước 21: bỏ qua cạnh 6⟶4 vì đỉnh 4 đã chốt*

Bước 23 — kết quả

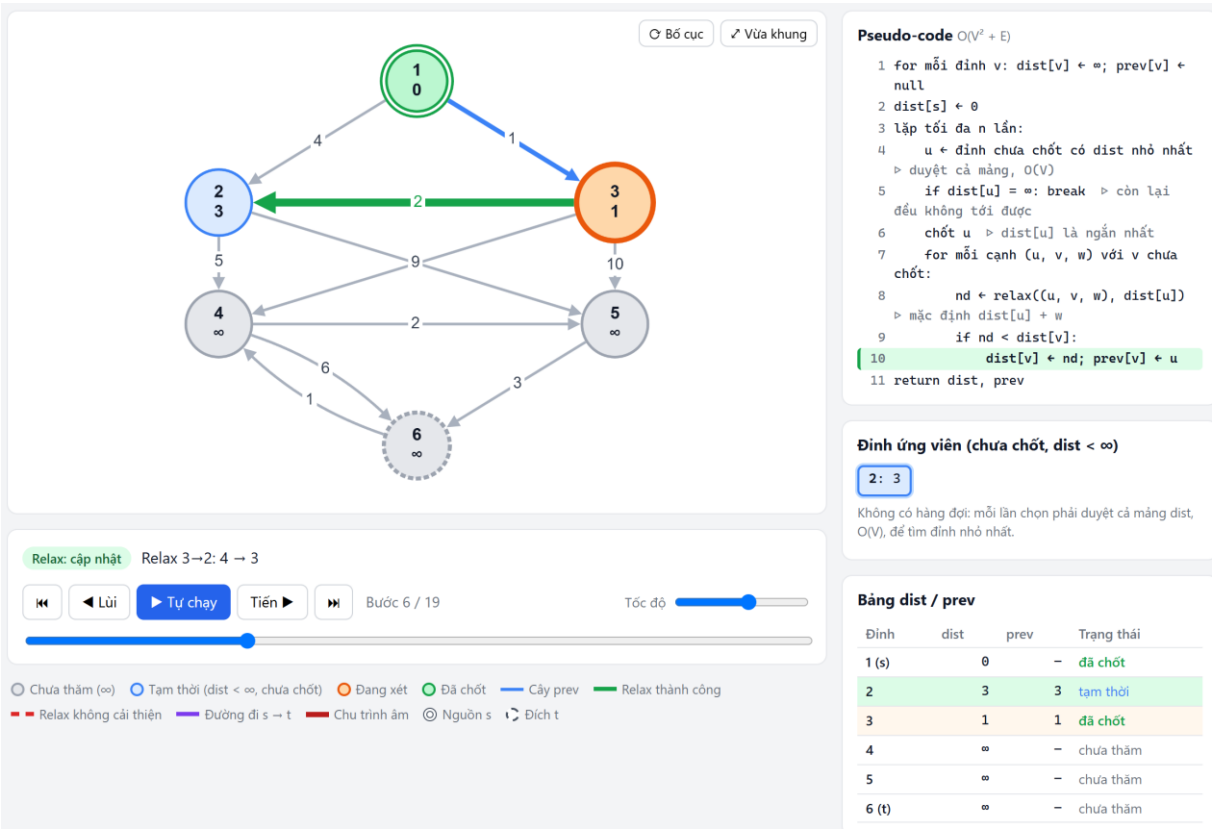
Hàng đợi rỗng, thuật toán kết thúc. Đường đi ngắn nhất 1 → 3 → 2 → 4 → 5 → 6 có độ dài 1 + 2 + 5 + 2 + 3 = 13, được tô tím.



Hình 5.4. *Bước 23: đường đi ngắn nhất 1 → 3 → 2 → 4 → 5 → 6, độ dài 13*

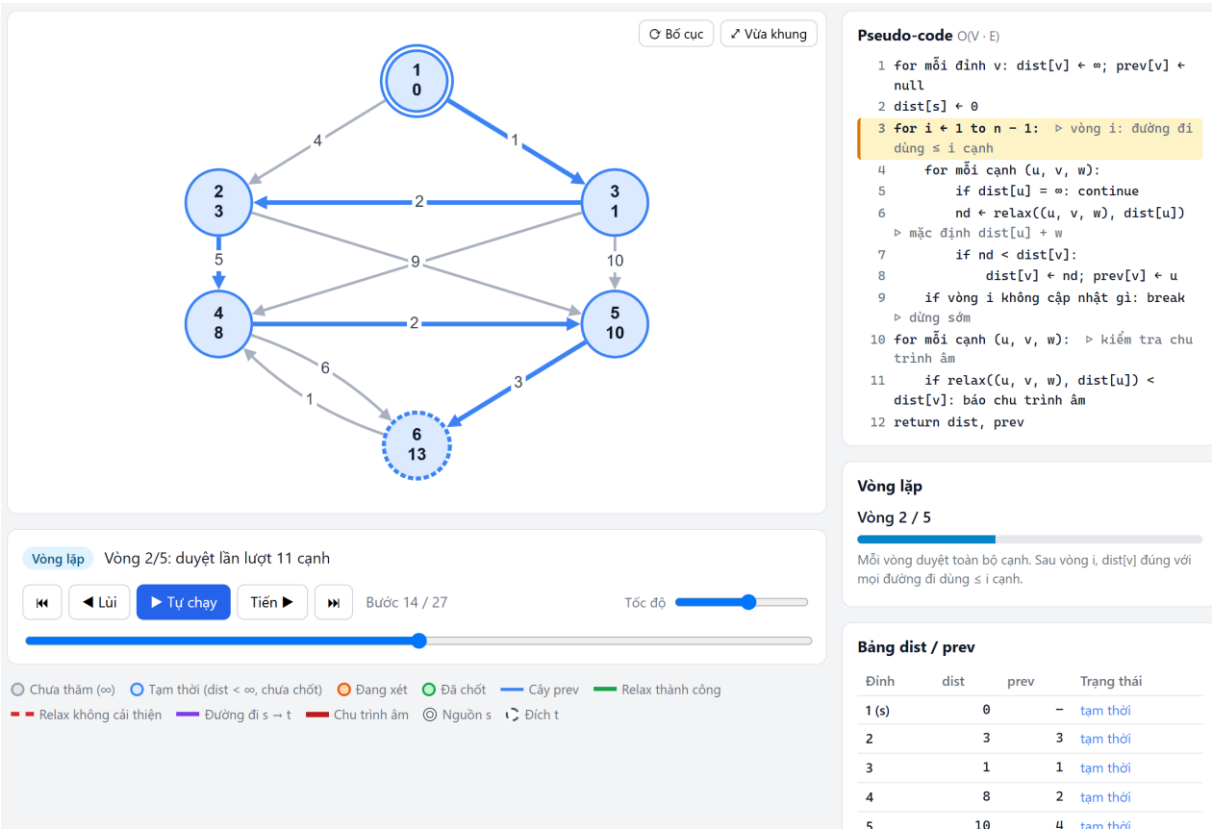
5.4. Cùng đồ thị với Dijkstra mảng và Bellman-Ford

Dijkstra (mảng) cho 19 bước: các đỉnh được chốt theo đúng thứ tự như bản heap (1, 3, 2, 4, 5, 6) nhưng không có bước "bỏ qua phần tử cũ", vì mỗi lần chọn đỉnh là duyệt lại cả mảng dist. Khung bên phải hiện danh sách đỉnh ứng viên thay cho heap.



Hình 5.5. *Dijkstra (mảng), bước 6: danh sách ứng viên thay cho hàng đợi ưu tiên*

Bellman-Ford cho 27 bước. Vì các cạnh trong input gần như đã theo thứ tự topo, vòng 1 đã tìm ra mọi khoảng cách đúng; vòng 2 duyệt lại 11 cạnh mà không cập nhật gì nên thuật toán **dừng sớm**, không cần chạy đủ V − 1 = 5 vòng. Kết quả trùng với Dijkstra.



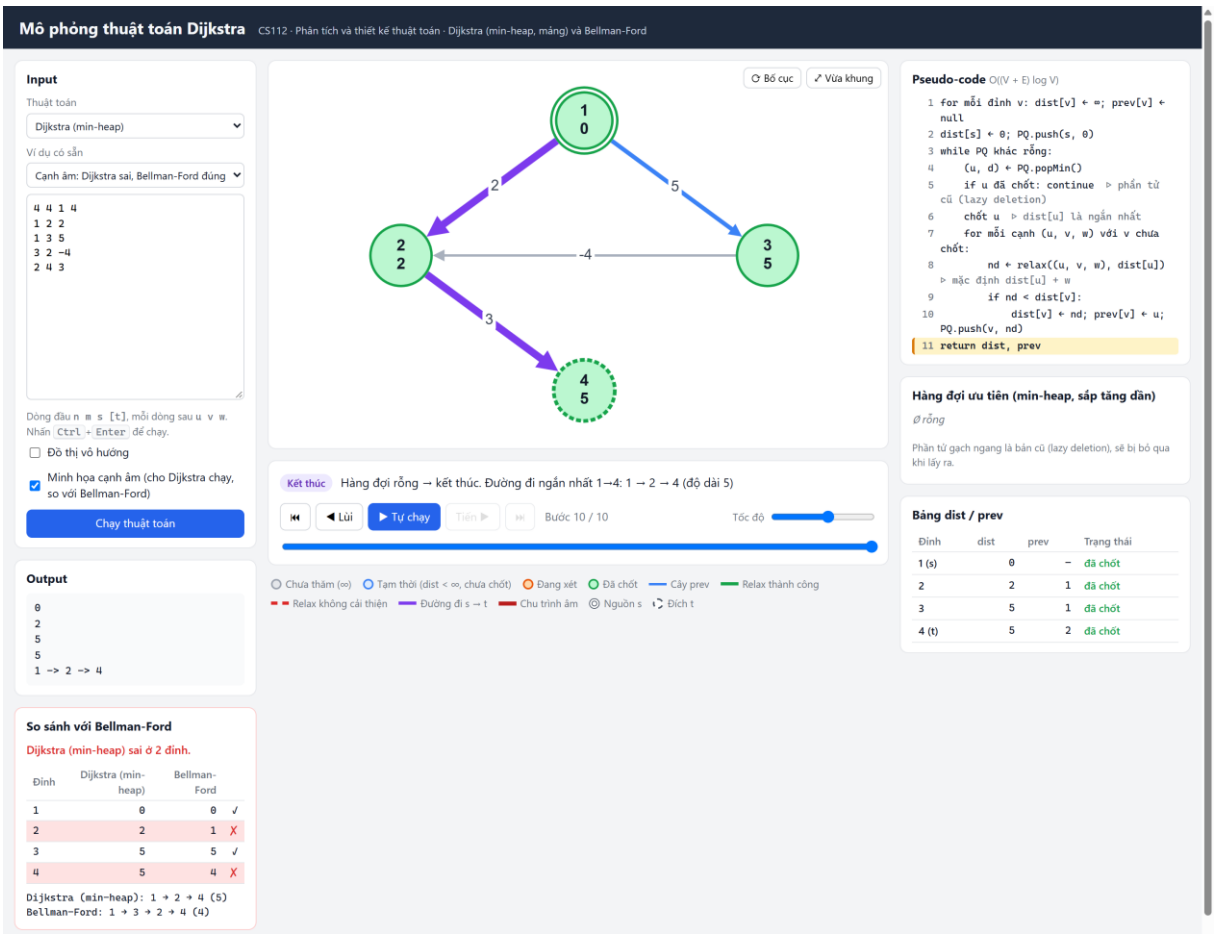
Hình 5.6. *Bellman-Ford, bước 14: bắt đầu vòng 2/5*

Bảng 5.2. *So sánh ba thuật toán trên ví dụ thống nhất*

Thuật toán	Số bước trace	Ghi chú
Dijkstra (min-heap)	23	4 bước bỏ qua phần tử cũ
Dijkstra (mảng)	19	6 lần duyệt mảng tìm đỉnh nhỏ nhất
Bellman-Ford	27	2 vòng, dừng sớm; 22 lần relax

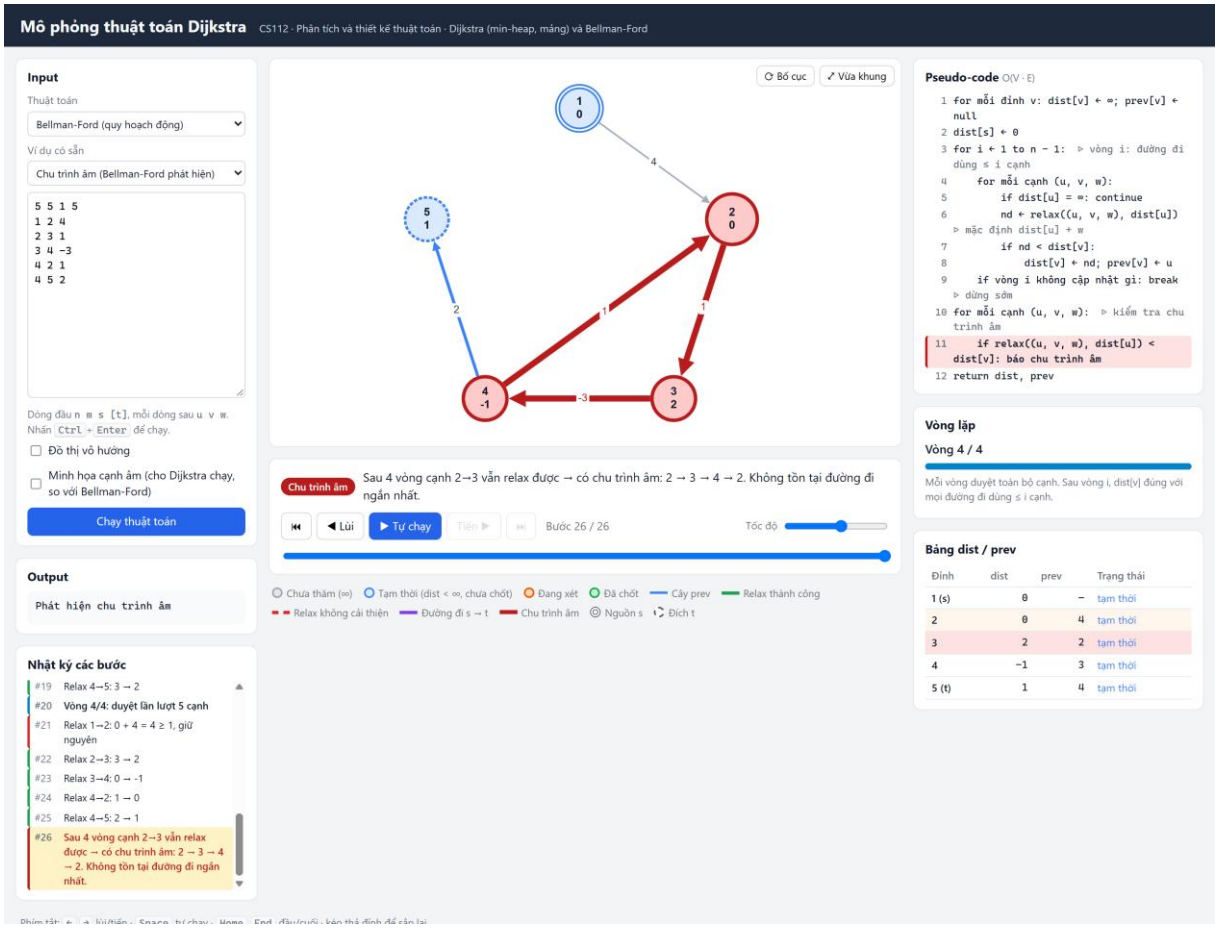
5.5. Cạnh âm và chu trình âm

Với ví dụ ở mục 2.5, khi bật **Minh họa cạnh âm**, Dijkstra vẫn chạy và tới bước cuối hiện bảng so sánh với Bellman-Ford: sai ở đỉnh 2 (2 thay vì 1) và đỉnh 4 (5 thay vì 4). Bước quyết định là bước 8: "Bỏ qua 3→2: đỉnh 2 đã chốt".



Hình 5.7. *Dijkstra sai trên đồ thị có cạnh âm; bảng so sánh với Bellman-Ford*

Với đồ thị có chu trình $2 \to 3 \to 4 \to 2$ tổng $1 + (-3) + 1 = -1$, sau $V - 1 = 4$ vòng cạnh 2→3 vẫn relax được. Bellman-Ford báo chu trình âm, tô đỏ chu trình và output `Phát hiện chu trình âm`.



Hình 5.8. *Bellman-Ford phát hiện và tô đỏ chu trình âm 2 → 3 → 4 → 2*

Chương 6. Kiểm thử

6.1. Chiến lược

- **Đáp án độc lập với thuật toán được kiểm tra:** vét cạn mọi đường đi đơn ($n \le 8$), Floyd-Warshall ($n \le 300$), Bellman-Ford có dừng sớm (đồ thị lớn). Khi sinh test, các nguồn đáp án được đối chiếu chéo với nhau; lệch là dừng.
- **Tái tạo được:** test sinh bằng `scripts/gen-tests.js` với bộ sinh số ngẫu nhiên mulberry32 có seed (seed = 20261018 + số thứ tự test).
- **Cả 3 cài đặt** chạy trên mọi test qua cùng hàm `solve` (chuỗi input → chuỗi output) như một chương trình nộp chấm.

6.2. Bộ testcase

Bảng 6.1. *Năm nhóm testcase (44 test)*

Nhóm	Số test	Nội dung
`nho`	11	Ví dụ thống nhất + 10 đồ thị ngẫu nhiên $n \le 8$; đáp án vét cạn.
`bien`	9	$n = 1$; $m = 0$; $s = t$; trọng số 0; trọng số 10^9 (dist tới $\sim 10^{12}$); khuyên và cạnh song song; không có t; t không tới được; $n = 10^5$ không cạnh.
`dac-biet`	9	Cạnh âm (Dijkstra phải báo lỗi); chu trình âm tới được / không tới được từ s; chu trình tổng 0; đồ thị đầy đủ; hình sao; lưới nhiều đường bằng nhau; nhiều phần tử cũ trong heap.
`lon`	4	$n = 10^5$, $m = 2 \cdot 10^5$ ngẫu nhiên; chuỗi 10^5 đỉnh $w = 10^9$ (dist $\approx 10^{14}$); đồ thị dày $n = 450$; lưới 220×220.
`khong-hop-le`	11	Input rỗng, thiếu số, chữ, số thực, $n = 0$, đỉnh ngoài phạm vi, w quá lớn, thiếu/thừa dòng cạnh, m quá lớn.

6.3. Cách chấm

- Các dòng khoảng cách phải khớp chính xác.
- Dòng đường đi: chấp nhận mọi đường hợp lệ từ s tới t có độ dài bằng dist[t], vì có thể có nhiều đường ngắn nhất (ví dụ test lưới mọi trọng số 1). So khớp chuỗi sẽ đánh trượt oan.
- Test có cạnh âm: Dijkstra (heap, mảng) đạt khi báo đúng lỗi cạnh âm; Bellman-Ford phải ra đáp án.
- Nhóm `khong-hop-le`: output phải đúng thông báo lỗi, ví dụ `Lỗi: Dòng 2: "a" không phải số nguyên (w).`

6.4. Kết quả

`node scripts/run-tests.js` chạy 44 test × 3 thuật toán: **128 đạt, 0 không đạt, 4 bỏ qua**. Bốn lượt bỏ qua là Dijkstra mảng trên các test $n \ge 48\,400$ ($O(V^2)$ quá chậm).

Bảng đầy đủ ở `4_Report/so-lieu/ket-qua-kiem-thu.md`; bảng dưới trích nhóm đồ thị lớn.

Bảng 6.2. *Kết quả nhóm đồ thị lớn (thời gian gồm cả đọc input, ghi output)*

Test	n / m	Heap	Mảng	Bellman-Ford
lon/01-thua-ngau-nhien	100 000 / 200 000	✓ 472.7 ms	— (n lớn)	✓ 1578.7 ms
lon/02-chuoi-trong-so-max	100 000 / 199 998	✓ 308.4 ms	— (n lớn)	✓ 361.4 ms
lon/03-day	450 / 200 000	✓ 231.3 ms	✓ 214.8 ms	✓ 332.9 ms
lon/04-luoi	48 400 / 192 720	✓ 242.4 ms	— (n lớn)	✓ 5075.1 ms

Ngoài bộ test chính thức, trong lúc cài đặt đã kiểm tra chéo 3000 đồ thị ngẫu nhiên ($n \leq 9$) với Floyd-Warshall: 3 thuật toán cho cùng dist; Bellman-Ford đúng với 387 ca có cạnh âm không chu trình âm và phát hiện đúng 566 ca chu trình âm. 0 ca sai. Test `lon/02` xác nhận độ chính xác số lớn: dist cuối 99 999 000 000 000 đúng ($< 2^{53}$).

6.5. Bộ test có bắt được lỗi không? (mutation test)

Để biết bộ test có thực sự phát hiện lỗi, đã **cố ý cài lỗi**: xóa dòng so sánh với con phải trong `siftDown` của min-heap (mục 4.2), rồi chạy lại `run-tests.js`. Kết quả: 5 test không đạt, đều ở bản heap — `dac-biet/06-day-du`, `dac-biet/09-nhieu-phan-tu-cu` và 3 test nhóm `lon`.

Điều đáng chú ý là **cả 11 test nhỏ ngẫu nhiên đều không phát hiện được lỗi này**, vì heap nhỏ ít khi rơi vào trường hợp con phải nhỏ hơn con trái. Bài học: cần nhóm test đặc biệt được thiết kế có chủ đích (nhiều phần tử cũ trong heap) và test lớn. Sau đó đã khôi phục mã và chạy lại 128/128 đạt.

Chương 7. Đánh giá hiệu năng

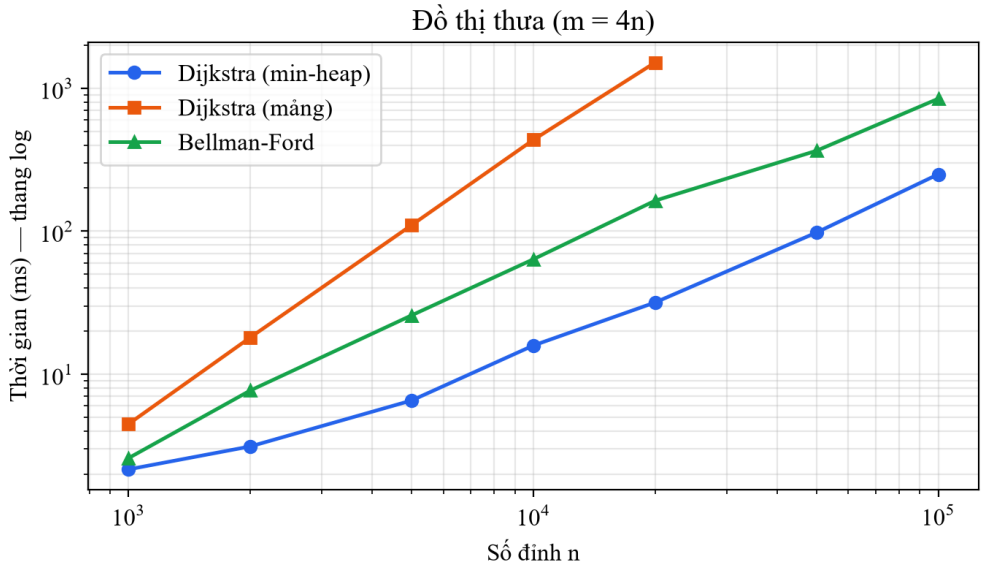
7.1. Thiết lập

Máy Intel Core i5-9400F 2.90 GHz, Node.js 24.20.0, seed 20261017. Đồ thị dựng sẵn, tắt ghi trace, chỉ đo phần thuật toán; mỗi ô lấy trung vị 5 lần chạy (1 lần nếu quá 1,5 giây), có khởi động JIT trước. Hai họ đồ thị: **thưa** ($m = 4n$, n tới 10^5) và **dày** ($m \approx n^2/2$, n tới 1000). Script: `scripts/benchmark.js`; số liệu: `4_Report/so-lieu/benchmark.md`, `.csv`.

7.2. Đồ thị thưa

Bảng 7.1. *Thời gian (ms) trên đồ thị thưa*

n	m	Heap	Mảng	Bellman-Ford
1 000	4 000	2.2	4.5	2.6
2 000	8 000	3.1	18.1	7.7
5 000	20 000	6.5	110.1	25.8
10 000	40 000	15.9	435.2	63.7
20 000	80 000	31.8	1513.7	163.6
50 000	200 000	97.8	—	365.0
100 000	400 000	250.0	—	843.9



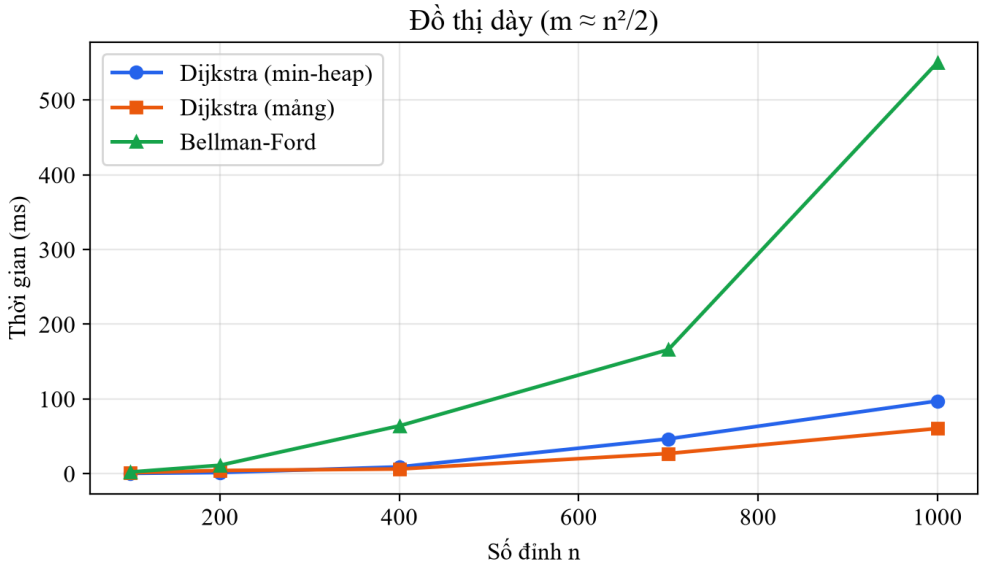
Hình 7.1. *Thời gian chạy trên đồ thị thưa (thang log – log)*

Khi n tăng 10 lần (2 000 → 20 000), bản mảng chậm đi khoảng 84 lần (18.1 → 1513.7 ms), đúng bậc $O(V^2)$ (100 lần); bản heap chậm đi khoảng 10 lần (3.1 → 31.8 ms), đúng bậc $O((V + E) \log V)$. Ở $n = 20\,000$, heap nhanh hơn mảng khoảng 48 lần. Số phép so sánh của bản mảng là $n(n + 1)/2$ (200 010 000 ở $n = 20\,000$), trong khi heap chỉ có 26 853 lần push.

7.3. Đồ thị dày

Bảng 7.2. *Thời gian (ms) trên đồ thị dày*

n	m	Heap	Mảng	Bellman-Ford
100	4 950	0.4	1.2	2.1
200	19 900	1.5	4.2	11.2
400	79 800	9.0	6.2	63.9
700	244 650	46.5	26.9	165.9
1 000	499 500	97.3	60.4	550.4



Hình 7.2. *Thời gian chạy trên đồ thị dày*

Trên đồ thị dày, từ khoảng n = 400 bản mảng vượt bản heap (ở n = 1000: 60.4 ms so với 97.3 ms), đúng như phân tích $O(V^2)$ so với $O(V^2 \log V)$ khi $E \approx V^2$. Với n nhỏ hơn, bản heap vẫn nhanh hơn do hằng số nhỏ và nhiều phần tử cũ chưa tích lũy.

7.4. Nhận xét

- Kết quả đo khớp với lý thuyết: **heap cho đồ thị thưa, mảng cho đồ thị dày**.
- Bellman-Ford trên đồ thị ngẫu nhiên chỉ cần 6–15 vòng nhờ dừng sớm (thay vì $V - 1$), nên chậm hơn Dijkstra chỉ vài lần chứ không tới hàng nghìn lần như cận $O(V \cdot E)$. Ngược lại, trên test lưới `lon/04` nó cần nhiều vòng và mất khoảng 5 giây so với 0,24 giây của heap.
- Số phần tử cũ trong heap (lazy deletion) trên đồ thị thưa khoảng 34 % số đỉnh, trên đồ thị dày tới gần 5 lần số đỉnh; vẫn nhỏ so với E nên không ảnh hưởng bậc độ phức tạp.
- Hạn chế: đo trên một máy, JavaScript có JIT và bộ thu gom rác nên số đo dao động; vì vậy dùng trung vị và chỉ kết luận về xu hướng.

Chương 8. Quá trình xây dựng và sử dụng AI

Chương này tường thuật lại quá trình làm đồ án, dựa trên `DEVLOG.md` (ghi sau mỗi yêu cầu, prompt chép nguyên văn) và lịch sử commit git. Mục tiêu là cho thấy rõ phần nào AI làm, phần nào sinh viên quyết định, và kết quả AI được kiểm chứng ra sao.

8.1. Công cụ và cách tổ chức làm việc

Bảng 8.1. *Công cụ sử dụng*

Công cụ	Model / phiên bản	Công đoạn
Claude trên claude.ai (Project môn học, có đính kèm slide và đề)	Claude Opus 5.5	Phân tích đề, lập kế hoạch, hỏi lý thuyết
Claude Code (CLI, chạy trong thư mục dự án)	Claude Opus 5.5	Viết code, testcase, script kiểm thử, kịch bản video, bản nháp report
Cytoscape.js	3.30.2	Vẽ đồ thị trên giao diện
Node.js	24.20.0 LTS	Server tĩnh, testcase, benchmark

Cách tổ chức: giai đoạn đầu dùng Claude trên claude.ai để phân tích đề và chốt phạm vi; kết quả được đúc kết thành hai file đặt ở thư mục dự án. `CLAUDE.md` ghi phạm vi, kiến trúc, cấu trúc trace, định dạng input/output và **quy tắc làm việc** (Claude Code tự đọc file này khi khởi động). `DEVLOG.md` là nhật ký. Quy tắc trong `CLAUDE.md` buộc Claude Code sau mỗi yêu cầu phải ghi một mục DEVLOG (prompt nguyên văn, tóm tắt, file sửa, vấn đề gặp phải), ghi lỗi vào "Nhật ký lỗi", commit git, và **để trống mục "Kiểm chứng của SV"** cho sinh viên tự viết.

8.2. Dòng thời gian

Bảng 8.2. *Các mốc chính (chi tiết đầy đủ trong DEVLOG.md)*

#	Ngày	Công đoạn	Prompt (nguyên văn, có rút gọn bằng "...")	Kết quả
#1	11/10	Phân tích đề	"Phân tích toàn bộ yếu cầu đố án sản phẩm cho môn này giúp tôi, dự tính của tôi làm mô phong thuật toán dijkstra"	Bóc tách 4 sản phẩm, yêu cầu "thống nhất"; gợi ý so sánh nhiều chiến lược; 3 mức phạm vi, khuyến nghị mức B.
#2	13/10	Kế hoạch, lý thuyết	"deadline là ngày 18/10" + câu hỏi về quy hoạch động và hành trình đường thư	Giải thích Bellman-Ford/Floyd; 3 cách giải hành trình theo lịch; gợi ý `relax(edge, t)`; thu hẹp phạm vi. SV quyết định giữ bài toán đơn giản.
#3	14/10	Thiết lập	"Giờ viết ra 1 file DEVLOG.md rồi tôi sẽ dùng claude cli ..."	Tạo `DEVLOG.md` và `CLAUDE.md`.
#4	14/10	Code lõi	"Đọc CLAUDE.md và DEVLOG.md. Tạo khung thư mục ..., viết parser input, min-heap và Dijkstra (heap) sinh trace ... Chưa làm giao diện."	Parser, graph, min-heap, Dijkstra heap, script in trace. Cài Node.js (hỏi SV trước). Phát hiện lỗi #1.
#5–#6	14/10	Tài liệu	"tạo giúp tôi 1 file readme ..."; "git remote add origin ..."	README hướng dẫn chạy; gắn remote GitHub.
#7–	15/10	Giao diện	"ủa hiện tại tôi phải chạy làm sao? sẽ	Giao diện web 3 cột, phát lại trace,

#	Ngày	Công đoạn	Prompt (nguyên văn, có rút gọn bằng "...")	Kết quả
#8			dựng 1 trang web chứ nhỉ"; "có làm luôn đi, cập nhật trên readme luôn"	server tĩnh; kiểm tra trên Chrome.
#9	15/10	Sửa lỗi môi trường	"chạy trên visual code bị:" + thông báo lỗi `npm.ps1`	Giải thích Execution Policy; hướng dẫn cách chạy khác; không tự đổi thiết lập bảo mật (lỗi #2).
#10–#11	15/10	Mở rộng	"chúng ta còn báo cáo kịch bản demo và demo phải không?"; "có làm bước 1 luôn đi"	Dijkstra mảng, Bellman-Ford, minh họa cạnh âm, chu trình âm; kiểm tra chéo 3000 đồ thị (lỗi #3, #4).
#12	17/10	Kiểm thử	"làm tiếp tục bước tiếp theo đi"	44 testcase, `run-tests.js`, benchmark, mutation test.
#13	17/10	Video	"soạn giúp tôi kịch bản demo đi, hướng dẫn step by step, để tôi vừa quay màn hình vừa nói"	Kịch bản 8 cảnh lấy số bước từ trace thật.
#14	18/10	Report	"vậy làm report giúp tôi đi, cái nào tôi dễ review và chỉnh sửa thì làm cái đó"	Bản nháp report này (Word), ảnh chụp tự động, biểu đồ.

Lịch sử commit git (mỗi chức năng một commit, message tiếng Việt) là bằng chứng tiến trình: từ "Khung dự án, parser, min-heap, Dijkstra heap sinh trace" (14/10) tới "Thêm kịch bản video demo" (17/10). Kế hoạch ban đầu (mục #10) dự kiến làm testcase ngày 16/10; thực tế trễ một ngày.

8.3. Lỗi phát hiện và cách xử lý

Bảng 8.3. *Nhật ký lỗi (từ DEVLOG)*

#	Lỗi	Phát hiện bằng	Nguyên nhân → cách sửa
1	Đồ thị mẫu không sinh bước "bỏ qua vì đỉnh đã chốt" như comment khẳng định	Chạy script in trace, đọc trace	Chọn cạnh chưa đủ → thêm cạnh `6 4 1`
2	`npm` bị chặn trong terminal VS Code	SV chạy thử trên máy	PowerShell Execution Policy → chạy `node scripts/serve.js` hoặc `npm.cmd`
3	Kiểm tra ngẫu nhiên gần như không sinh ca "cạnh âm, không chu trình âm" (1/3000)	Đếm số ca theo loại	Bộ sinh LCG lấy `% k` dùng bit thấp chu kỳ ngắn → đổi sang mulberry32 (387/3000)
4	Ví dụ chu trình âm: các cạnh chồng lên nhau	Chụp màn hình Chrome headless	Bố cục cây xếp đỉnh thẳng hàng → bố cục vòng tròn

Lỗi #3 là ví dụ cho thấy kết quả "0 ca sai" có thể đánh lừa: phép kiểm tra chéo đầu tiên vẫn "đạt" nhưng gần như không kiểm tra trường hợp cạnh âm. Chỉ khi đếm số ca theo từng loại mới thấy bộ sinh ngẫu nhiên có vấn đề.

8.4. Kết quả của AI được kiểm chứng như thế nào

Các kiểm chứng tự động đã thực hiện trong quá trình làm (ghi trong DEVLOG):

- Mọi thuật toán được đối chiếu với đáp án **độc lập** (vét cạn, Floyd-Warshall), không lấy Dijkstra làm đáp án cho chính nó.
- Mutation test (mục 6.5) để kiểm tra chính bộ test.

- Giao diện được kiểm tra bằng trình duyệt thật (Claude in Chrome, sau đó Chrome headless): đếm số bước, đọc output, kiểm tra console không lỗi.
- Số liệu trong report, kịch bản video được sinh từ trace và file kết quả thật, không chép tay.

Phần sinh viên tự kiểm chứng (SV viết — ví dụ: chạy tay Dijkstra trên giấy với đồ thị 6 đỉnh và so với 23 bước trên giao diện; đọc lại code phần nào; giữ / sửa / bỏ đề xuất nào của AI và vì sao):

[SV viết đoạn này. Gợi ý: nội dung các mục "Kiểm chứng của SV" trong DEVLOG.]

8.5. Phân công giữa sinh viên và AI

Bảng 8.4. *Phân công*

Việc	Sinh viên	AI
Chọn đề, phạm vi, hạn nộp	Chọn Dijkstra; chốt hạn 18/10; chọn phạm vi; quyết định để hành trình đường thư làm hướng phát triển	Phân tích đề, đề xuất 3 mức phạm vi
Quy ước dự án	[duyệt / sửa CLAUDE.md thế nào]	Soạn `CLAUDE.md`, `DEVLOG.md`
Mã nguồn	Ra yêu cầu từng bước, chạy thử trên máy, báo lỗi npm; [đọc lại / sửa phần nào]	Viết code, README, tự kiểm tra
Testcase, benchmark	[...]	Sinh test có seed, script chấm, benchmark
Video	Quay màn hình và thuyết minh	Soạn kịch bản
Report	[đọc lại, sửa, viết phần kiểm chứng và nhận xét]	Bản nháp, ảnh chụp, biểu đồ

8.6. Nhận xét về việc dùng AI

[SV tự viết: AI giúp được gì, chỗ nào AI sai hoặc phải sửa, bài học rút ra. Một số sự kiện có thể dùng: AI tự phát hiện lỗi #1 và #3 khi kiểm tra; AI hỏi trước khi cài Node.js và không tự đổi Execution Policy; lịch làm bị trễ 1 ngày; quy tắc CLAUDE.md giúp DEVLOG được ghi đều.]

Chương 9. Kết luận và hướng phát triển

9.1. Kết quả đạt được

- Chương trình web mô phỏng từng bước Dijkstra (min-heap) với pseudo-code, hàng đợi, bảng dist/prev, đồ thị tô màu; tiến/lùi/tự chạy.
- Đủ phần mở rộng: Dijkstra mảng, Bellman-Ford, minh họa cạnh âm với bảng so sánh, phát hiện chu trình âm.
- 44 testcase có đáp án độc lập, 128/128 lượt chạy đạt; mutation test cho thấy bộ test phát hiện được lỗi trong heap.
- Benchmark khớp lý thuyết: heap thắng trên đồ thị thưa (48 lần ở n = 20 000), mảng thắng trên đồ thị dày từ n ≈ 400.
- Report, video và testcase dùng chung một đồ thị; quá trình làm được ghi đầy đủ trong DEVLOG.

9.2. Hạn chế

- Giao diện chỉ vẽ tốt đồ thị ≤ 30 đỉnh.
- Ở bước "bỏ qua phần tử cũ", đỉnh bị bỏ qua vẫn tô màu "đang xét" (câu giải thích đã nói rõ).
- Chưa cài decrease-key / Fibonacci heap để so sánh với lazy deletion.
- Benchmark chỉ đo trên một máy.

9.3. Hướng phát triển

- **Hành trình đường thư theo lịch chuyến:** thay `relax(edge, t)` bằng hàm "giờ đến sớm nhất" theo bảng giờ chuyến; Dijkstra phụ thuộc thời gian vẫn đúng nhờ tính chất FIFO. Kiến trúc hiện tại đã chuẩn bị sẵn điểm mở rộng này.
- A* với heuristic khoảng cách, chạy trên bản đồ thật.
- So sánh decrease-key với lazy deletion; hiển thị heap dạng cây.

Tài liệu tham khảo

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022 — Chương 22 (Single-Source Shortest Paths).

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs", *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[3] R. Bellman, "On a routing problem", *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[4] L. R. Ford Jr., "Network Flow Theory", RAND Corporation, P-923, 1956.

[5] M. Franz et al., "Cytoscape.js 2023 update: a graph theory library for visualization and analysis", *Bioinformatics*, vol. 39, no. 1, 2023.

[6] [Slide bài giảng môn CS112 — tên tài liệu, giảng viên]